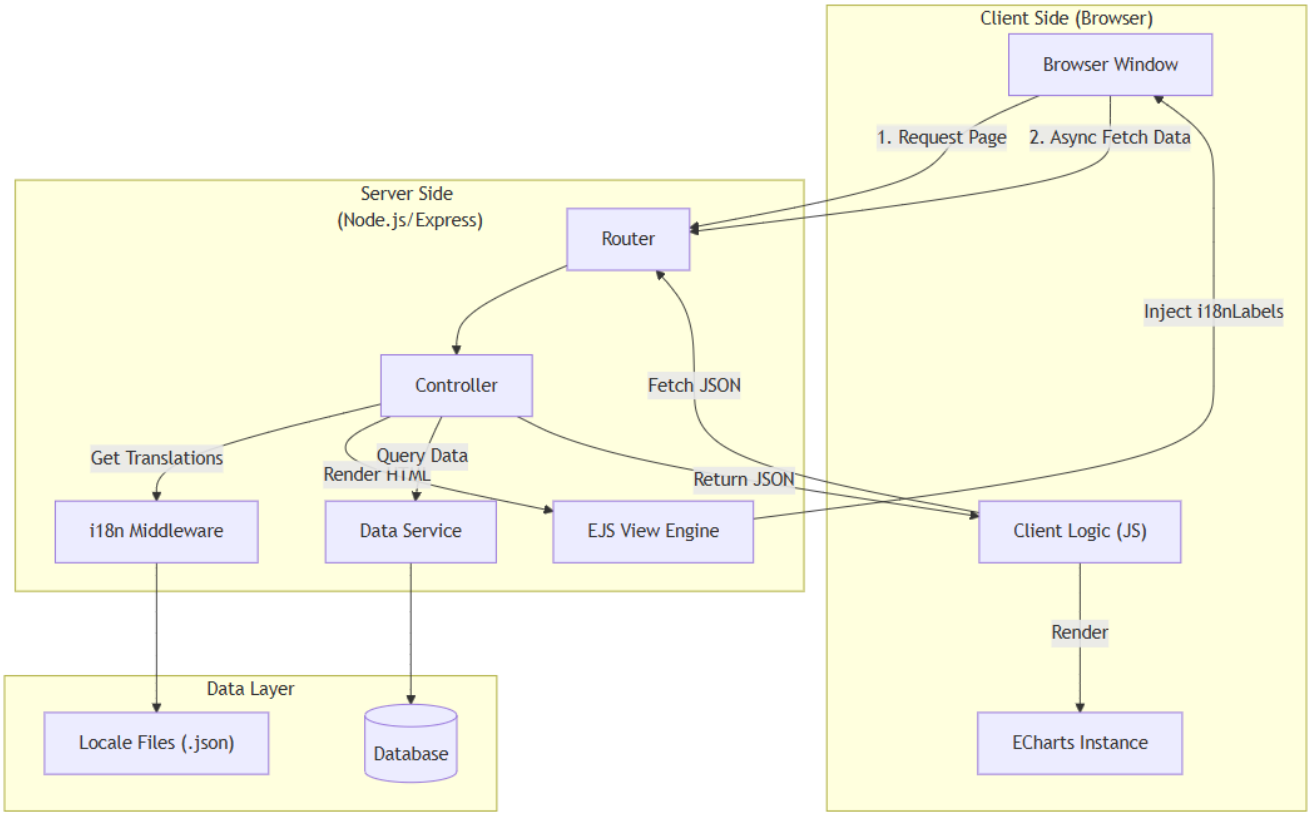
System Design Structure (系統設計架構)

本文件描述 **TY Data Visualization** 專案的整體系統設計架構，採用經典的 MVC (Model-View-Controller) 模式，結合後端渲染 (SSR) 與前端動態圖表 (Client-side Rendering) 的混合架構。

1. 系統架構概觀 (System Architecture)




```
graph TD
    subgraph Client ["Client Side (Browser)"]
        Browser[Browser Window]
        JS["Client Logic (JS)"]
        Chart[ECharts Instance]
    end

    subgraph Server ["Server Side (Node.js/Express)"]
        Route[Router]
        Ctrl[Controller]
        i18n[i18n Middleware]
        ViewEng[EJS View Engine]
        Service[Data Service]
    end

    subgraph Data [Data Layer]
        DB[(Database)]
        Locales["Locale Files (.json)"]
    end
```

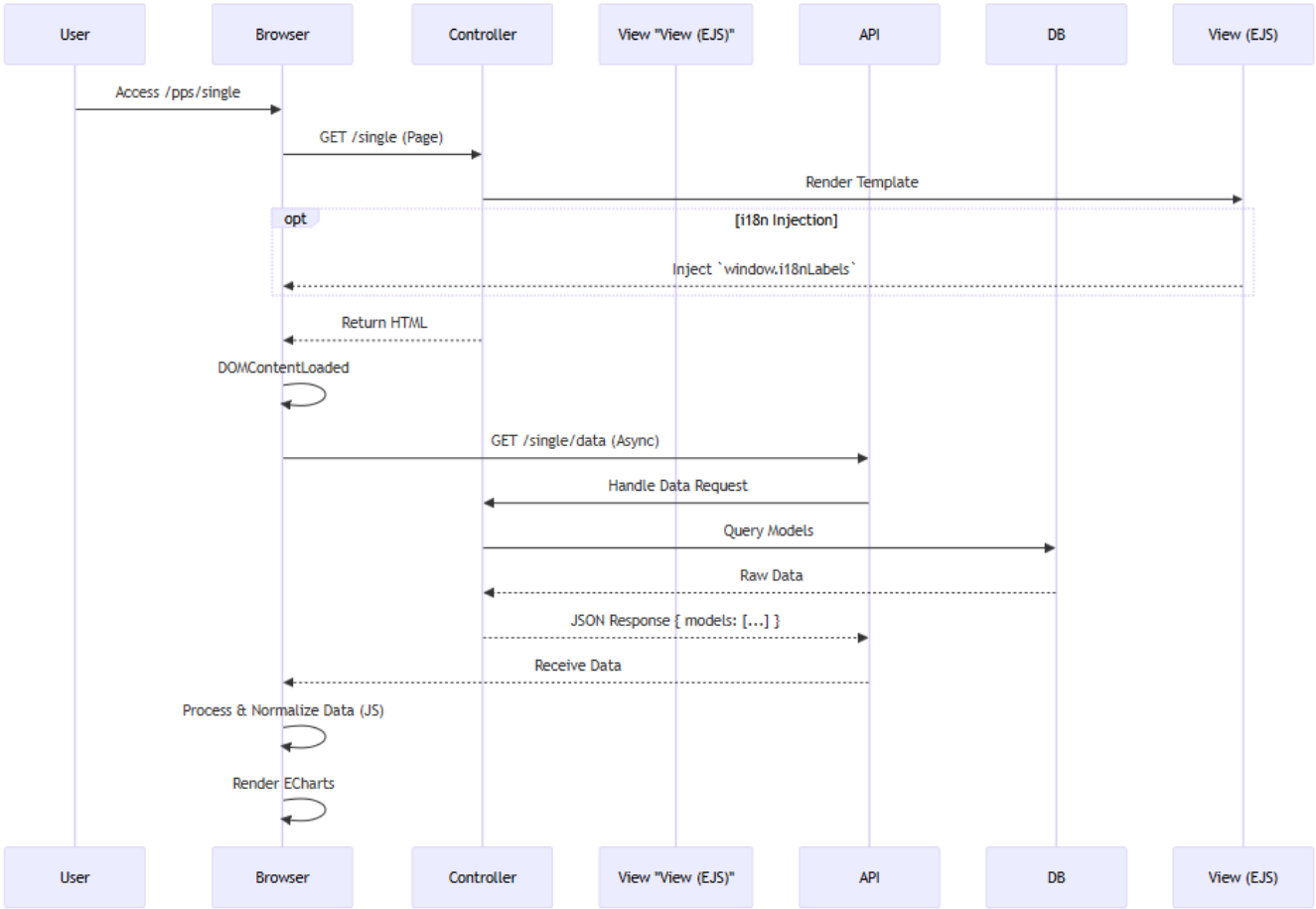
```
    %% Flow
    Browser -->|1. Request Page| Route
    Browser -->|2. Async Fetch Data| Route

    Route --> Ctrl
    Ctrl -->|Get Translations| i18n
    i18n --> Locales

    Ctrl -->|Render HTML| ViewEng
    ViewEng -->|Inject i18nLabels| Browser

    Ctrl -->|Query Data| Service
    Service --> DB

    JS -->|Fetch JSON| Route
    JS -->|Render| Chart

    Ctrl -->|Return JSON| JS
```

---

2. 模組設計模式 (Module Design Pattern)

每個業務模組 (如 `pps-single`, `biscuit`, `psmax`) 都遵循一致的 **Controller-Page-Data** 模式。

2.1 請求處理流程 (Request Processing Flow)



```
sequenceDiagram
    participant User
    participant Browser
    participant Controller
    participant View "View (EJS)"
    participant API
    participant DB

    %% Page Load Phase
    User->>Browser: Access /pps/single
    Browser->>Controller: GET /single (Page)
    Controller->>View (EJS): Render Template

    opt i18n Injection
        View (EJS)-->>Browser: Inject `window.i18nLabels`
    end

    Controller-->>Browser: Return HTML

    %% Client Fetch Phase
    Browser->>Browser: DOMContentLoaded
    Browser->>API: GET /single/data (Async)
    API->>Controller: Handle Data Request
    Controller->>DB: Query Models
    DB-->>Controller: Raw Data
    Controller-->>API: JSON Response { models: [...] }

    %% Rendering Phase
    API-->>Browser: Receive Data
    Browser->>Browser: Process & Normalize Data (JS)
    Browser->>Browser: Render ECharts
```

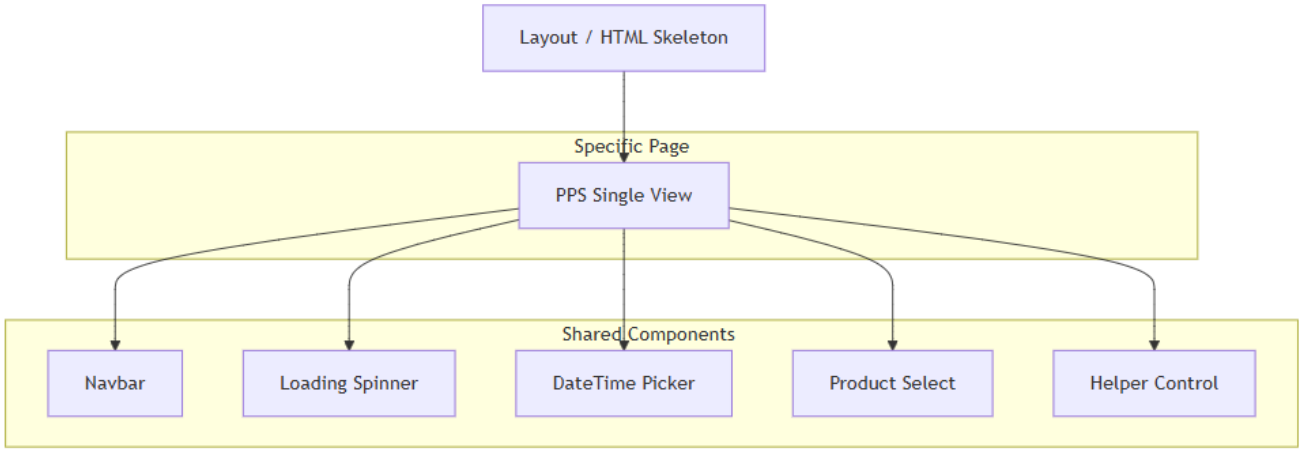
---

3. 資料層級架構 (Component Layer)

3.1 檔案結構對應 (File Structure Map)

Layer	Responsibility	Example File
Route	定義 URL 端點，分派請求至 Controller	`routes/pps.js`
Controller	處理業務邏輯，協調 Model 與 View	`controllers/ppsController.js`
View	定義 HTML 結構，負責畫面佈局與靜態資源引入	`views/pps-single.ejs`
Client JS	處理前端互動、數據輪詢、ECharts 渲染	`views/js/pps-single.js`
Locales	儲存多語系翻譯字串	`locales/en.json`

3.2 視圖組裝策略 (View Assembly)




```
graph TD
    Layout[Layout / HTML Skeleton]

    subgraph Partials ["Shared Components (Partials)"]
        Nav[Navbar]
        Loading[Loading Spinner]
        Picker[DateTime Picker]
        Dropbox[Product Select]
        Helper[Helper Control]
    end

    subgraph Page [Specific Page]
        PPS[PPS Single View]
    end

    Layout --> PPS
    PPS --> Nav
    PPS --> Loading
    PPS --> Picker
    PPS --> Dropbox
    PPS --> Helper
```

---

4. 關鍵技術決策 (Key Technical Decisions)

1. **混合渲染 (Hybrid Rendering)**:
    - **HTML**: 使用 EJS 進行伺服器端渲染 (SSR)，利用 `include` 複用元件 (Partials)。
    - **Data**: 使用 Fetch API 非同步獲取 JSON，減輕初次載入負擔，並支援即時輪詢。
    - **i18n**: 採用 "Backend Injection" 模式，由後端解析語系並注入前端變數。
2. **前端狀態管理 (Client State)**:

- 不依賴大型框架 (React/Vue)，使用原生 JS (`var`, `const`) 管理簡單狀態 (如 `timeLeft`, `autoRefresh`, `models`)。
- 利用 `echarts` 實例自身作為主要的 View State 呈現者。

3. **樣式架構 (CSS Architecture)**:

- 使用 Bootstrap 作為基礎 Grid System 與元件庫。
- 模組化 CSS (`/css/pps-single.css`) 覆蓋特定頁面樣式。